
Deloitte SQL Agent

Installation Guide

Windows 11 VM setup via SharePoint ZIP distribution

Audience	Deloitte users installing the agent on the approved Windows 11 VM
Platform	Windows 11, PowerShell, OpenRouter, Python 3.14, uv
Version date	April 2, 2026

1 Purpose

This document explains how to install and run the Deloitte SQL Agent on a Deloitte-provided Windows 11 virtual machine. The agent is distributed as a ZIP file via SharePoint.

2 Prerequisites

Before you begin:

- Use the Deloitte Windows 11 VM.
- Connect the VM to **Telekom WLAN**, not **DeloitteNet**.
- Make sure you can download files from SharePoint and browse to public websites.
- This guide assumes PowerShell is available.

This guide does not explain how to provision the VM itself. Use the Deloitte VM onboarding tutorial:

Deloitte VM onboarding tutorial

3 Step 1: Install uv

uv is the Python package manager used by this project. On Deloitte machines, prefer the official WinGet package.¹

Open **PowerShell** and run:

```
winget install --id=astral-sh.uv -e
```

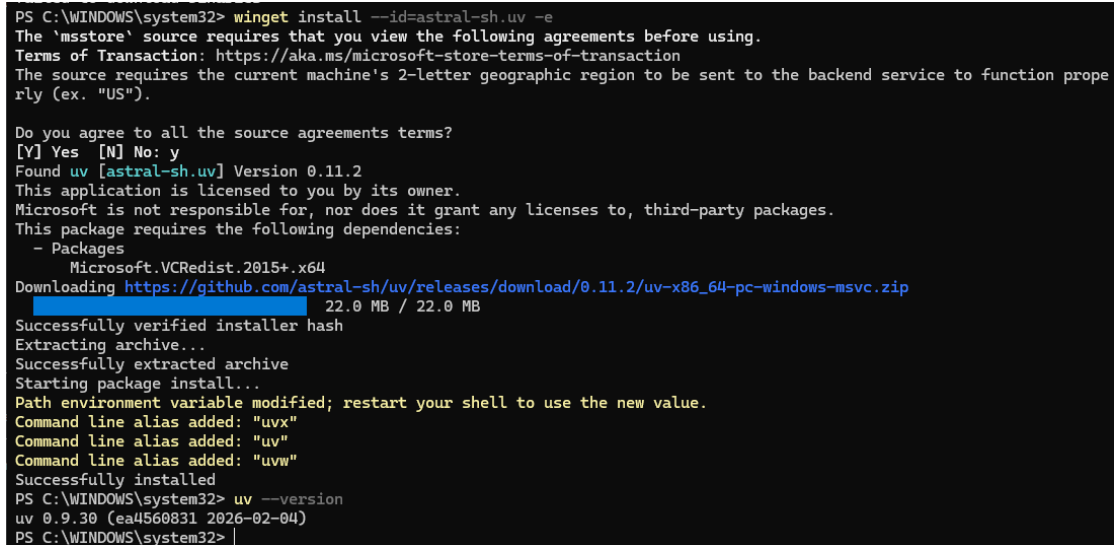
Close PowerShell and open it again, then verify the installation:

```
uv --version
```

¹<https://docs.astral.sh/uv/getting-started/installation/>

If WinGet is unavailable, you can use the official Astral PowerShell installer as an alternative. At the time this guide was written, this installer was often blocked in the Deloitte setup even on Telekom WLAN, so it should not be the primary path:

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```



```
PS C:\WINDOWS\system32> winget install --id=astral-sh.uv -e
The 'msstore' source requires that you view the following agreements before using.
Terms of Transaction: https://aka.ms/microsoft-store-terms-of-transaction
The source requires the current machine's 2-letter geographic region to be sent to the backend service to function properly (ex. "US").

Do you agree to all the source agreements terms?
[Y] Yes [N] No: y
Found uv [astral-sh.uv] Version 0.11.2
This application is licensed to you by its owner.
Microsoft is not responsible for, nor does it grant any licenses to, third-party packages.
This package requires the following dependencies:
- Packages
  Microsoft.VCRedist.2015+.x64
Downloading https://github.com/astral-sh/uv/releases/download/0.11.2/uv-x86_64-pc-windows-msvc.zip
22.0 MB / 22.0 MB
Successfully verified installer hash
Extracting archive...
Successfully extracted archive
Starting package install...
Path environment variable modified; restart your shell to use the new value.
Command line alias added: "uvx"
Command line alias added: "uv"
Command line alias added: "uvw"
Successfully installed
PS C:\WINDOWS\system32> uv --version
uv 0.9.30 (ea4560831 2026-02-04)
PS C:\WINDOWS\system32>
```

Figure 1: uv installation via WinGet and the verification step in PowerShell.

4 Step 2: Install Python 3.14 with uv

This project requires Python 3.14 or newer. The requirement is defined in the project metadata.

Install Python 3.14 from PowerShell:

```
uv --native-tls python install 3.14
```

Why -native-tls?

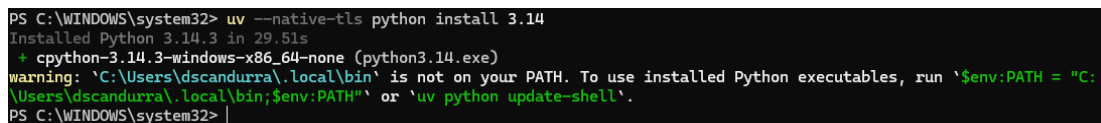
Use -native-tls on Deloitte VMs so uv reads certificates from the Windows certificate store. This is the recommended way to make uv work in managed environments with custom trust settings.^a

^a<https://docs.astral.sh/uv/reference/cli/>

You can verify that uv sees the installed interpreter:

```
uv --native-tls python list
```

Reference: official uv Python installation guide.²



```
PS C:\WINDOWS\system32> uv --native-tls python install 3.14
Installed Python 3.14.3 in 29.51s
+ cpython-3.14.3-windows-x86_64-none (python3.14.exe)
warning: 'C:\Users\dscandurra\.local\bin' is not on your PATH. To use installed Python executables, run '$env:PATH = "C:\Users\dscandurra\.local\bin;$env:PATH"' or 'uv python update-shell'.
PS C:\WINDOWS\system32>
```

Figure 2: Python 3.14 installation with uv -native-tls python install 3.14.

²<https://docs.astral.sh/uv/guides/install-python/>

5 Step 3: Download and Extract the ZIP from SharePoint

1. Open SharePoint.
2. Navigate to **Qualitätsmanagement** → **Testdatenmanagement** → **dump subset**.
3. Download the ZIP file **agent.zip**.
4. Right-click the ZIP file and choose **Extract All**.
5. Extract it to a simple working folder, for example:

`C:\Users\<your-user>\Projects\deloitte-sql-agent`

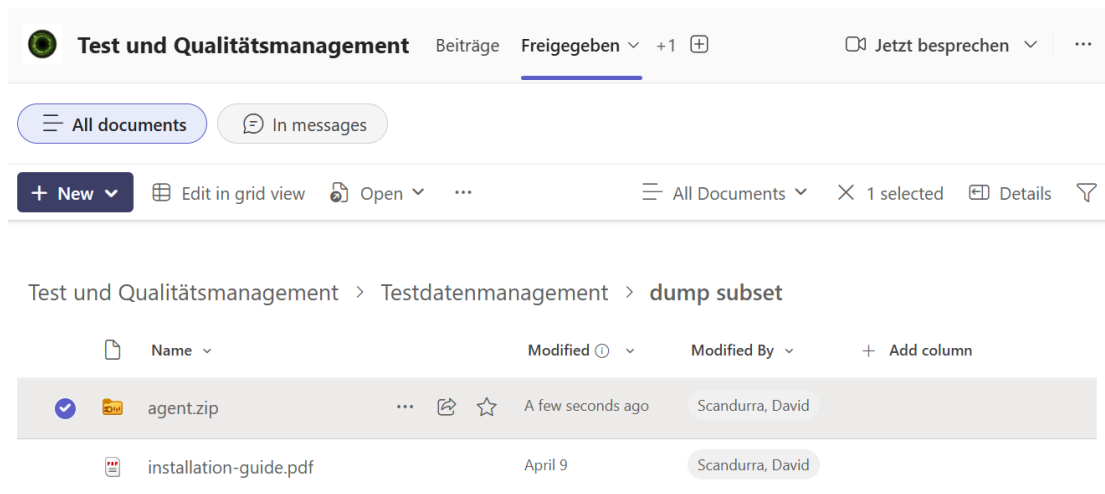


Figure 3: SharePoint download page for the Deloitte SQL Agent ZIP package.

6 Step 4: Install the Project Dependencies

Open PowerShell in the extracted project folder and run:

```
cd C:\Users\<your-user>\Projects\deloitte-sql-agent
uv --native-tls sync
```

This creates the local virtual environment and installs the project dependencies from the lockfile.

```

PS C:\Users\dscandurra\PycharmProjects\dump-reducer> uv --native-tls sync
Using CPython 3.14.3
Creating virtual environment at: .venv
Resolved 19 packages in 57ms
Prepared 3 packages in 1.21s
Installed 18 packages in 2.06s
+ certifi==2026.1.4
+ charset-normalizer==3.4.4
+ colorama==0.4.6
+ faker==40.4.0
+ idna==3.11
+ iniconfig==2.3.0
+ markdown-it-py==4.0.0
+ mdurl==0.1.2
+ packaging==26.0
+ pluggy==1.6.0
+ psycpg==3.3.2
+ psycpg-binary==3.3.2
+ pygments==2.19.2
+ pytest==9.0.2
+ requests==2.32.5
+ rich==14.3.2
+ tzdata==2025.3
+ urllib3==2.6.3
PS C:\Users\dscandurra\PycharmProjects\dump-reducer>

```

Figure 4: Successful dependency installation with `uv --native-tls sync`.

7 Step 5: Create an OpenRouter Token

The SQL Agent calls OpenRouter for model inference, so each user needs an OpenRouter API key (token).

5.1 Sign in

Open the OpenRouter sign-in page:

<https://openrouter.ai/sign-in>

5.2 Set up payment if required

Depending on your OpenRouter account or workspace, you may need to add payment information before you can create and use an API key.

1. Open your OpenRouter account or workspace settings.
2. Add payment details if required by your setup.
3. Return to the API key area after payment is configured.

5.3 Open the keys page

After signing in, open the keys page:

<https://openrouter.ai/settings/keys>

At the time this document was written on April 2, 2026, this URL redirects to the workspace key-management page after login.

5.4 Create and copy the token

1. Click the button to create a new API key.
2. Give the key a clear name, for example `Deloitte SQL Agent`.
3. Copy the new key immediately.

OpenRouter user interface note

The OpenRouter website may change over time. If buttons or menu labels look slightly different, the overall process should still be similar: sign in, set up payment if needed, then create an API key.

OpenRouter references:

- [OpenRouter Quickstart](#)
- [OpenRouter API key documentation](#)

Important: Treat the token like a password. Do not send it by email or save it in screenshots.

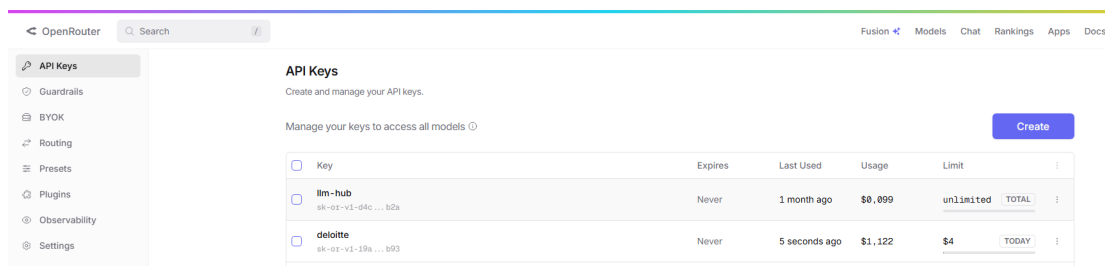


Figure 5: OpenRouter during payment and token setup.

8 Optional Step: Create a Sample SQLite Database for a Test Run

If you want to validate the installation before using a real source database, you can generate a local SQLite sample database inside the project folder.

Run:

```
uv --native-tls run python generate_sample_dump.py '  
--db-filename sample_dump.db '  
--scenario support '  
--scale tiny '  
--hidden-reference-mode light '  
--skip-sql-dump
```

This creates a file named `sample_dump.db` in the current directory. The `tiny` scale is intended for a quick setup check and keeps the test run fast.

For additional fixture-generator options and larger scenarios, see `docs/usage.md` in the project ZIP.

9 Step 6: Run the SQL Agent

Run the agent from the project folder. The example below uses the sample SQLite database created in the optional step above. If you already have a real database, replace the value of `-db-url`.

```
uv --native-tls run python main.py '
  --db-url sqlite:///C:/Users/<your-user>/Projects/deloitte-sql-agent/sample_dump.db '
  --target-rows 1000 '
  --out subset.sql '
  --model moonshotai/kimi-k2-thinking '
  --api-key sk-or-v1-REPLACE_ME '
  --no-verify-ssl
```

Why `-no-verify-ssl`?

The SQL Agent itself uses Python HTTP requests for the OpenRouter call. In the Deloitte VM setup, run the agent with `-no-verify-ssl`. Only use this on the approved VM and trusted Telekom WLAN setup.

Notes:

- `uv --native-tls` is used for the project launcher.
- `-no-verify-ssl` is passed to the SQL Agent itself.
- The tutorial uses `-api-key` directly on the command line.
- The output file `subset.sql` will be written into the current directory unless you provide a different path.
- For the full planner command reference and additional arguments, see `docs/usage.md` in the project ZIP.

Optional alternative: environment variable

If you do not want to pass the key on the command line every time, you can set `OPENROUTER_API_KEY` in PowerShell and omit `-api-key` from the run command.



```
Good, I have the schema. Now I need to get stats for high-value tables to understand data distribution. Let me identify the high-value tables based on row estimates:

High-value tables (row estimates):
- users: 480
- user_roles: 720
- user_sessions: 970
- products: 440
- product_tags: 1200
- price_book_entries: 980
- order_items: 1305
- orders: 720
- shipments: 570
- shipment_items: 1087
- invoices: 720
- invoice_items: 1305
- payments: 668
- tickets: 300
- ticket_comments: 550
- tasks: 321
- task_comments: 482
- addresses: 288
- suppliers: 80
- supplier_contacts: 112
- warehouses: 40
- inventory_bills: 120
- carts: 240
- cart_items: 470

Let me get stats for some key tables to understand the data distribution better. I'll start with users, orders, products, and tenants.

sql_get_stats called with: {'table': 'users'}
Result: {'table': 'users', 'total_rows': 480, 'stats': [{'column': 'id', 'n_distinct': 480, 'top_values': [[1, 1], [2, 1], [3, 1], [4, 1], [5, 1]], 'null_frac': 0.0}, {'column': 'tenant_id', 'n_distinct': 10, 'top_values': [[10, 40], [9, 40], [8, 40], [7, 40], [6, 40]]}, {'column': 'department_id', 'n_distinct': 40, 'top_values': [[20, 20], [40, 10], [30, 10], [10, 10], [16, 10]]}, {'column': 'team_id', 'n_distinct': 50, 'top_values': [[10, 10], [40, 10], [15, 15], [14, 14], [30, 13]]}, {'column': 'cost_center_id', 'n_distinct': 30, 'top_values': [[10, 20], [4, 20], [19, 19], [2, 19], [10, 10]]}, {'column': 'manager_user_id', 'n_distinct': 200, 'top_values': ...}]}
```

Figure 6: Successful first run of the SQL Agent in PowerShell.

10 Troubleshooting

uv command is not found

Close PowerShell and open it again. If that still does not help, reinstall uv and verify with:

```
uv --version
```

TLS or certificate errors during uv commands

- Confirm that the VM is connected to **Telekom WLAN**, not DeloitteNet.
- Make sure the command includes `-native-tls`.
- If the issue continues, contact the Deloitte support contact for the VM environment.

TLS or certificate errors during the SQL Agent run

- Confirm that the run command includes `-no-verify-ssl`.
- Confirm that the VM is using **Telekom WLAN**.
- Retry the command in a fresh PowerShell window.

OpenRouter authentication failed

- Check that the value passed to `-api-key` is correct.
- If you use `OPENROUTER_API_KEY` instead, verify that the variable is set correctly.
- Generate a new token in OpenRouter if necessary.

11 Quick Reference

```
winget install --id=astral-sh.uv -e
uv --native-tls python install 3.14
cd C:\Users\
```